

CS455: Introduction to Software Engineering

Lecture 1

Priyanka Bagade



Software Systems

- Software systems are abstract and intangible.
- They are not constrained by the properties of materials, governed by physical laws, or by manufacturing processes.
- However, because of the lack of physical constraints, software systems can quickly become extremely complex, difficult to understand, and expensive to change.
- Different types of software systems, from simple embedded systems to complex, worldwide information systems
- It is pointless to look for universal notations, methods, or techniques for software engineering because different types of software require different approaches.

Software failures

- Increasing demands: Systems have to be built and delivered more quickly; systems should be upgraded with new capabilities
- Low expectations:
 - relatively easy to write computer programs without using software engineering methods and techniques.
 - Developers do not usually use software engineering methods in their everyday work. Consequently, their software is often more expensive and less reliable than it should be.

Need for better software engineering education and training to address this problem.

Software Engineering

- Professional software
 - intended for use by someone apart from its developer
 - usually developed by teams rather than individuals
 - maintained and changed throughout its life.
- Software engineering
 - intended to support professional software development, rather than individual programming.
 - includes techniques that support program specification, design, and evolution, none of which are normally relevant for personal software development.
 - consists of a number of separate programs and configuration files that are used to set up these programs
 - include system documentation to describe the structure of the system; user documentation, which explains how to use the system, and websites for users to download recent product information.

Question	Answer
What is software?	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What are the attributes of good software?	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable, and usable.
What is software engineering?	Software engineering is an engineering discipline that is concerned with all aspects of software production.
What are the fundamental software engineering activities?	Software specification, software development, software validation, and software evolution.
What is the difference between software engineering and computer science?	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
What is the difference between software engineering and system engineering?	System engineering is concerned with all aspects of computer-based systems development including hardware, software, and process engineering. Software engineering is part of this more general process.
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times, and developing trustworthy software.
What are the costs of software engineering?	Roughly 60% of software costs are development costs; 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best software engineering techniques and methods?	While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.
What differences has the Web made to software engineering?	The Web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

Types of Software Products

- Generic products – e.g. software for PCs such as databases, word processors, drawing packages, and project-management tools. It also includes so-called vertical applications designed for some specific purpose such as library information systems, accounting systems, or systems for maintaining dental records.
- Customized products - software include control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.

Software Quality Attributes

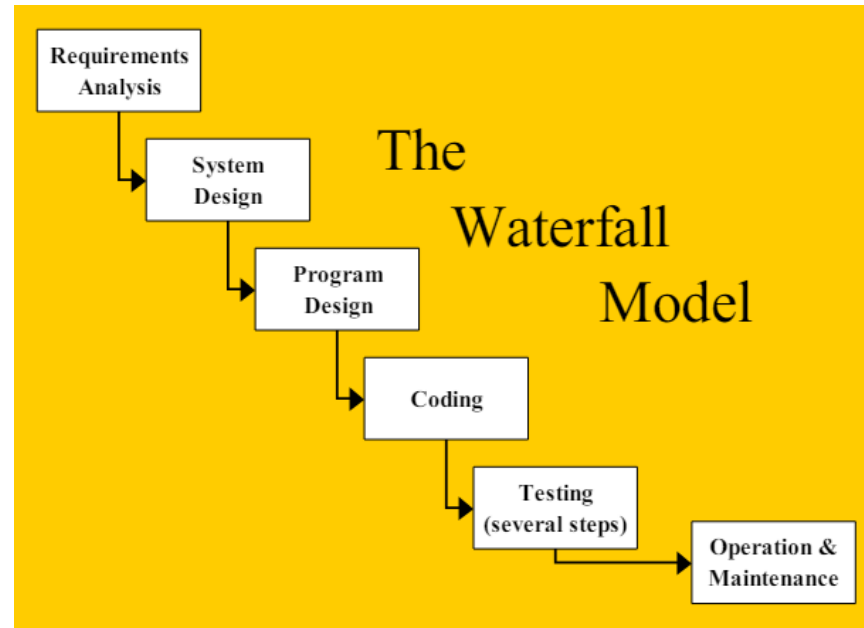
Product characteristics	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security, and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilization, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable, and compatible with other systems that they use.

Software development lifecycle & activities – Software Processes

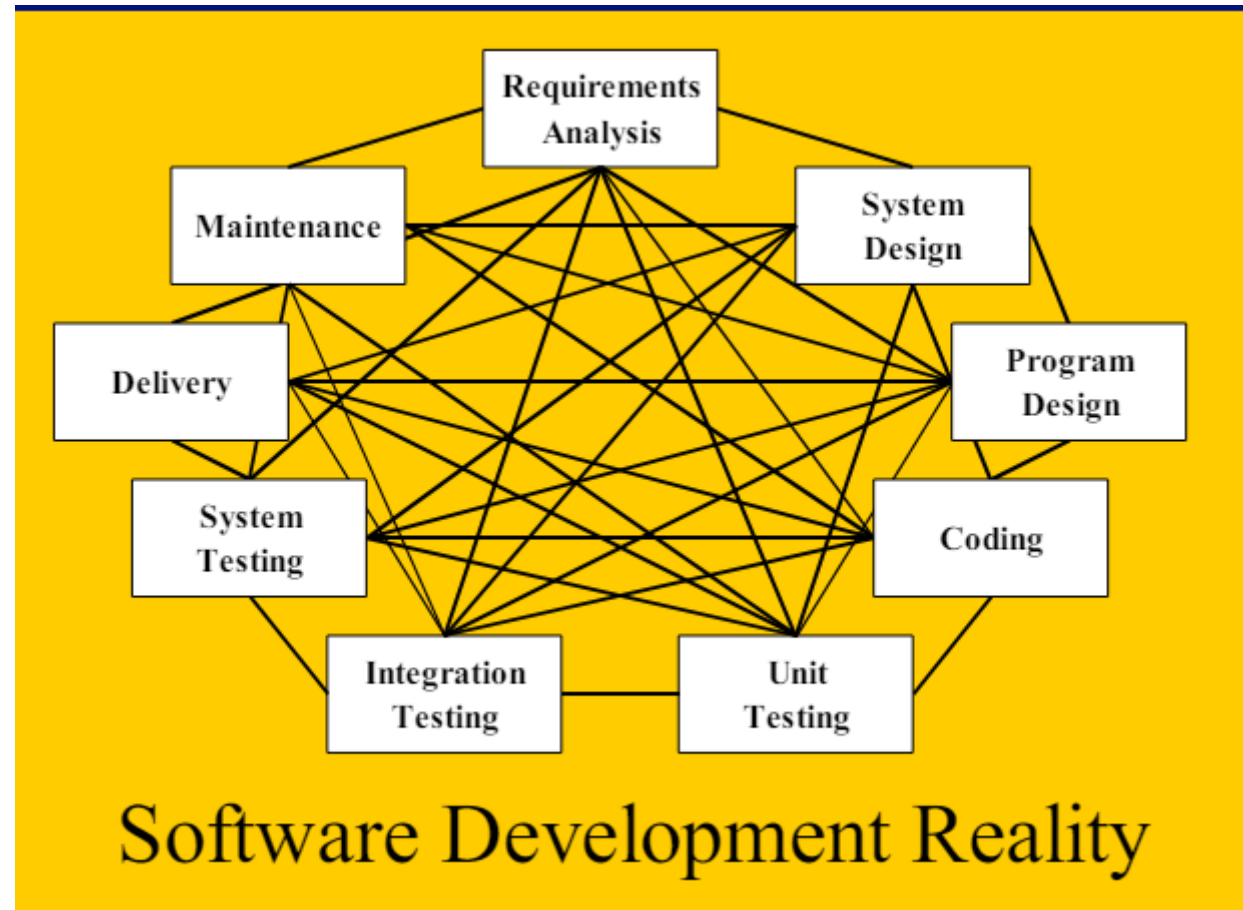
- Planning
- Domain & Technical analysis
- Requirements specification (functional & non-functional)
- Design & Architecture
- User experience / interface design and testing
- Programming (development)
- Debugging
- Testing
- Documentation
- Deployment, Maintenance, Change, updates
- Project management (staffing, negotiations, budget)

Software development lifecycle models

- How activities are executed in a project



SDLC Models



SDLC models

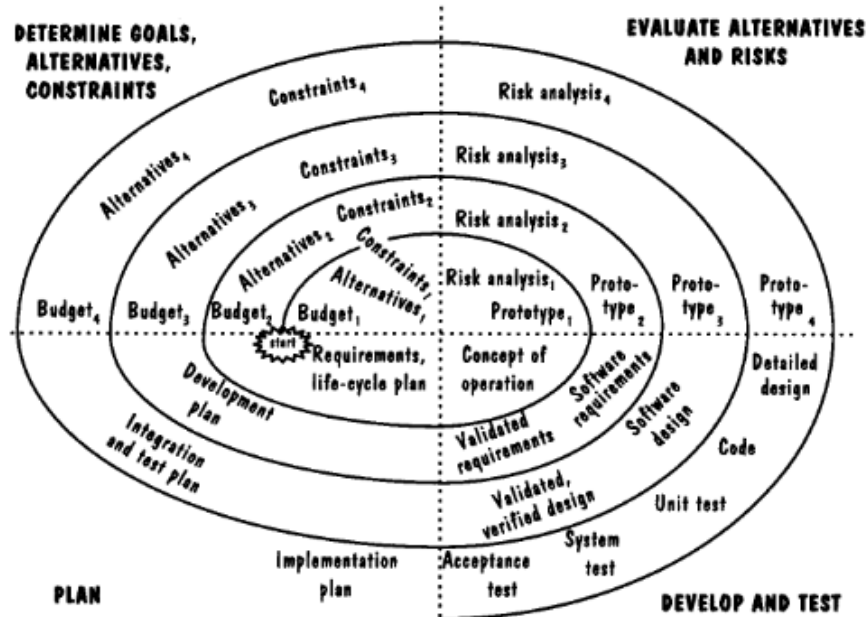
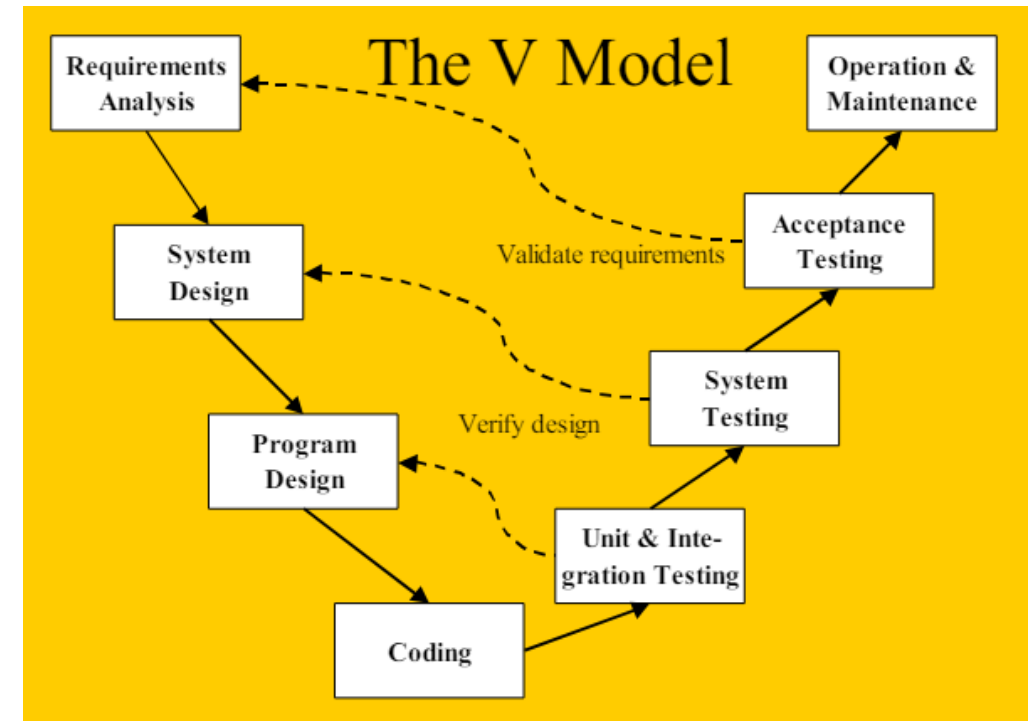
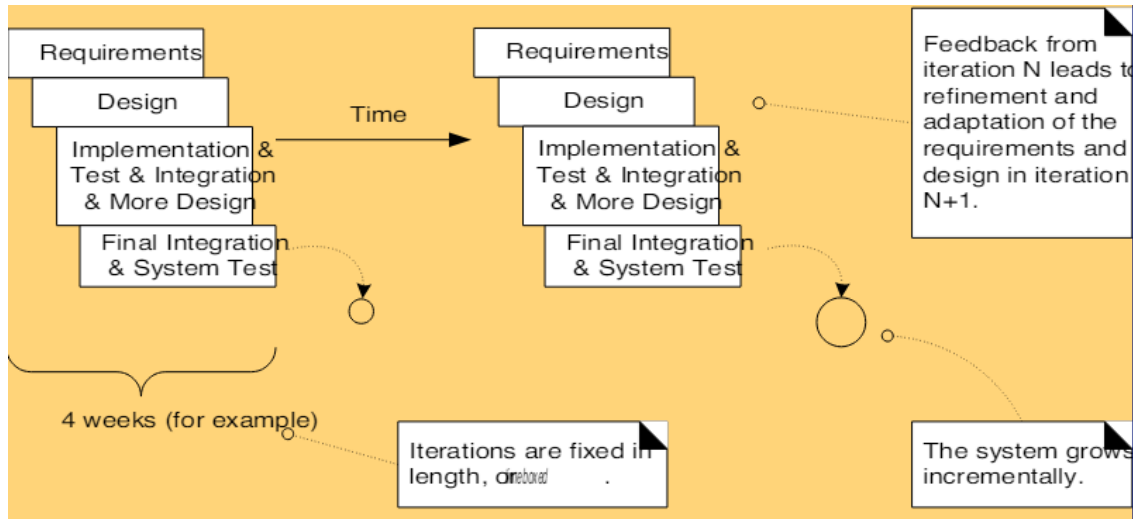


FIGURE 2.10 The spiral model.

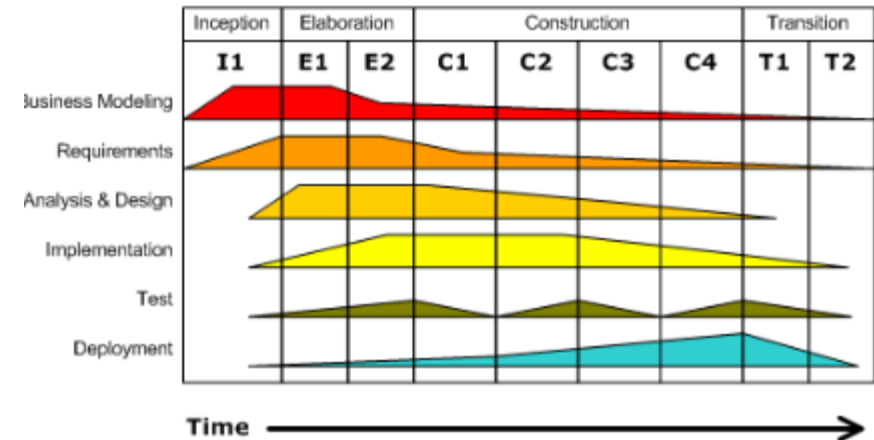


SDLC models



Iterative Development

Business value is delivered incrementally in time-boxed cross-discipline iterations.



Newer models

- Agile (Scrum)
- Extreme Programming
- Kanban
- Lean
- We won't do any of this in CS455!

What is in CS455?

- Software health
 - Good coding practices – hmm... isn't it the job of LLMs now???
 - Design, architecture
 - Tests
 - CI/CD pipelines; version control
- Software resilience
 - Scale, Performance, Robustness to failures
- Some Project management
 - Time, budget, scope, process measurements
- Tools for SE
 - How to build software like adults (Debug, Profile, etc.)

References

- Software engineering: [Software Engineering 9th Edition by Ian Sommerville](#)